



CS240 Algorithm Design and Analysis

Lecture 3

Greedy Algorithms (Cont.) Divide and Conquer

Qingying Hao
Spring 2026
2026.03.09



Last Time – What you need to know

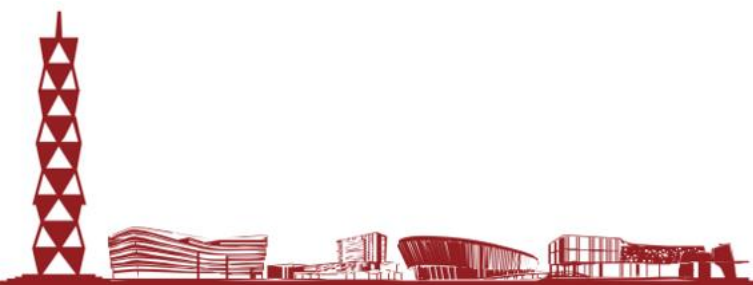


- Basic idea
 - A greedy algorithm always makes the choice that **looks best at the moment** and adds it to the current partial solution
 - Greedy algorithms don't always yield optimal solutions, but when they do, they're usually the **simplest and most efficient** algorithms available
 - Make the locally optimal choice at each step
- Algorithms
 - Interval Scheduling
 - Choose the job with the **earliest finish time**
 - Scheduling to Minimize Lateness
 - Choose the job with the **earliest deadline**





Clustering





Clustering



Photos, documents, micro-organisms



- **Clustering.** Given a set U of n objects labeled $p_1, \dots, p_n$, partition into clusters s.t. objects in different clusters are far apart.



e.g., a large number of corresponding pixels whose intensities differ over some threshold

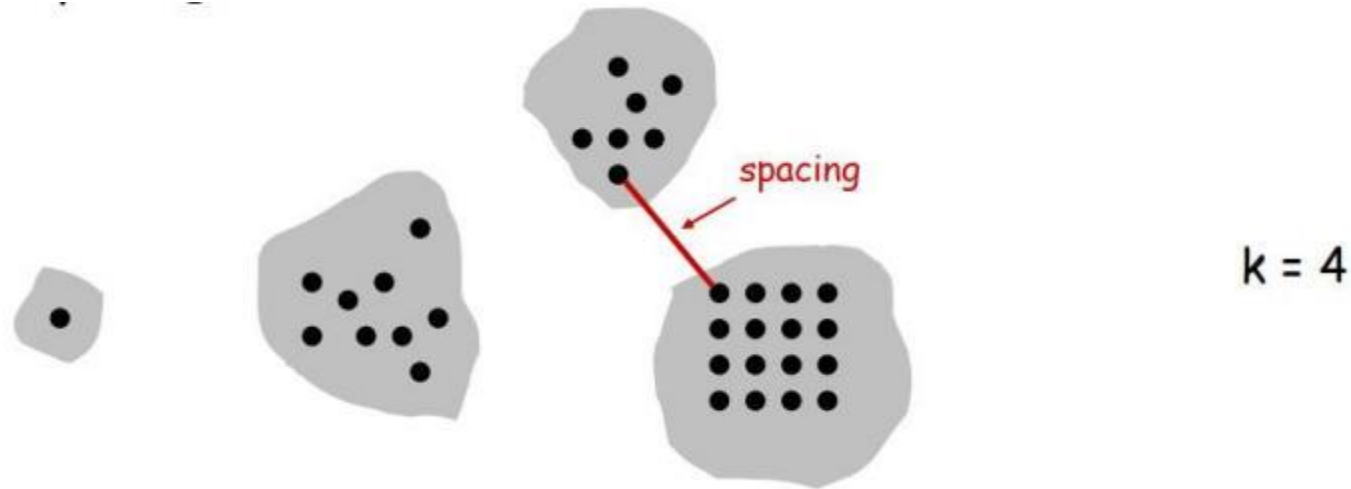
- **Fundamental problem.** Divide into clusters so that points in different clusters are far apart.
 - Routing in mobile ad hoc networks
 - Identify patterns in gene expression
 - Document categorization for web search
 - Similarity searching in medical image databases
 - Skycat: cluster 10^9 sky objects into stars, quasars, galaxies





Clustering of Maximum Spacing

- **k-clustering.** Divide objects into k non-empty groups
- **Distance function.** Assume it satisfies several natural properties
 - $d(p_i, p_j) = 0$ iff $p_i = p_j$ (identity of indiscernible)
 - $d(p_i, p_j) \geq 0$ (nonnegativity)
 - $d(p_i, p_j) = d(p_j, p_i)$ (symmetry)
- **Spacing.** Min distance between any pair of points in different clusters
- **Clustering of maximum spacing.** Given an integer k , find a k -clustering of maximum spacing





Greedy Clustering Algorithm



- **Single-link k-clustering algorithm.**

- Create n clusters, one for each object
- Find the closest pair of objects such that each object is in a different cluster; add an edge between them and merge the two clusters
- Repeat $n-k$ times until there are exactly k clusters

- **Key observation.** This procedure is precisely Kruskal's algorithm (except we stop when there are k connected components)

- **Remark.** Equivalent to finding an MST and deleting the $k-1$ most expensive edges





Greedy Clustering Algorithm: Analysis



- **Theorem.** Let C^* denote the clustering $C^*_1, \dots, C^*_k$ formed by deleting the $k-1$ most expensive edges of a MST. C^* is a k -clustering of **max spacing**.
 - spacing = min distance between different clusters $spacing(C) = \min_{i \neq j} dist(p_i, p_j)$

A —1— B —2— C —8— D —12— E

If $k = 3$, delete $k-1 = 2$ most expansive edges

A —1— B —2— C D E

Cluster 1: {A,B,C}

Cluster 2: {D}

Cluster 3: {E}

C-D = 8, D-E = 12

$\min(8, 12) = 8$

Spacing equals the $(k - 1)$ -**st** most expensive edge in the MST.

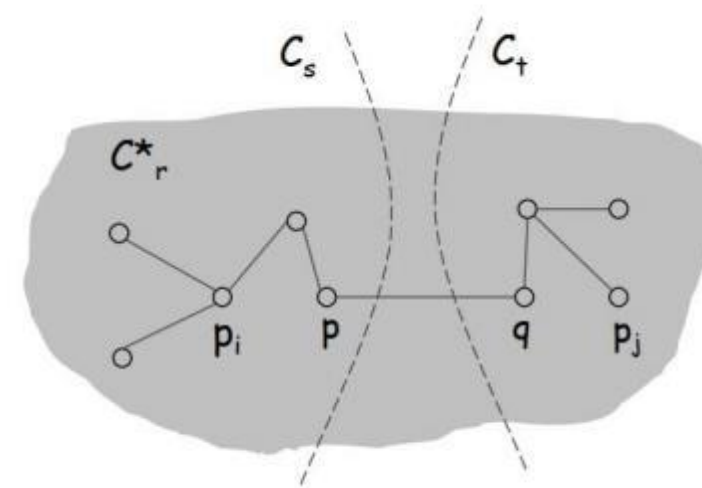




Greedy Clustering Algorithm: Analysis

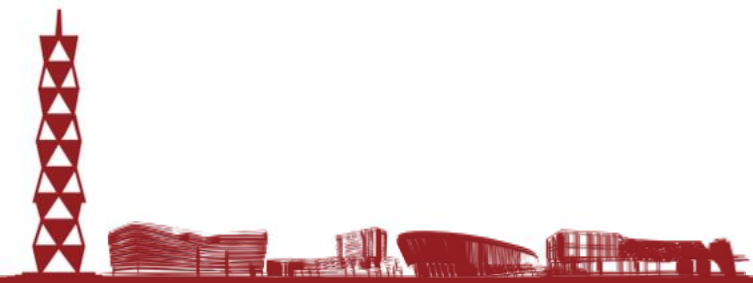


- **Theorem.** Let C^* denote the clustering $C^*_1, \dots, C^*_k$ formed by deleting the $k-1$ most expensive edges of a MST. C^* is a k -clustering of **max spacing**.
 - spacing = min distance between different clusters $spacing(C) = \min_{i \neq j} dist(p_i, p_j)$
- Pf. Let C denote some **other** clustering $C_1, \dots, C_k$ $spacing(C) \leq spacing(C^*)$
 - The spacing of C^* is the length d^* of the **(k-1)st** most expensive edge in MST $spacing(C^*) = d^*$
 - Let p_i, p_j be in the same cluster in C^* , say C^*_r , but different clusters in C , say C_s and C_t
 - Some edge (p, q) on $p_i \text{ --- } p_j$ path in C^*_r spans two different clusters in C .
 - All edges on $p_i \text{ --- } p_j$ path have length $\leq d^*$ since Kruskal chose them
 - $spacing(C) \leq d^*$ since p and q are in different clusters





Optimal Caching

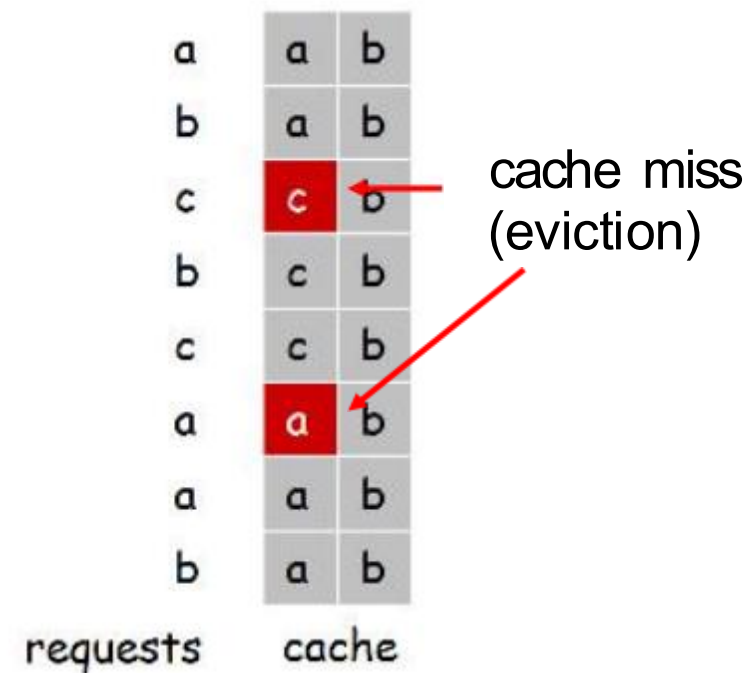




Optimal Offline Caching



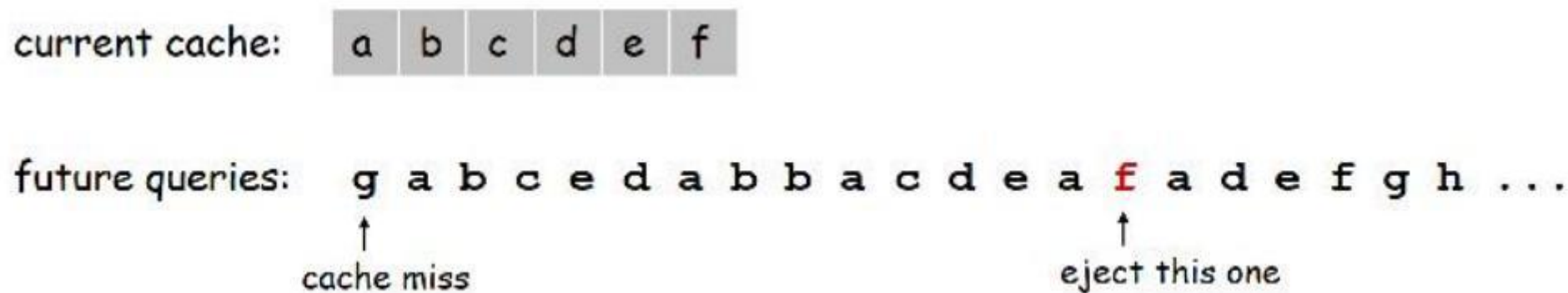
- **Caching**
 - Cache with capacity to store k items
 - Sequence of m item requests $d_1, d_2, \dots, d_m$
 - **Cache hit:** item already in cache when requested
 - **Cache miss:** item not already in cache when requested; must bring requested item into cache, and evict some existing item, if full
- **Applications.** CPU, RAM, hard drive, web, browser, ...
- **Goal.** Eviction schedule that minimizes number of evictions
 - Ex: $k = 2$, initial cache = ab , requests: a, b, c, b, c, a, a, b
- **Optimal eviction schedule:** 2 evictions





Optimal Offline Caching: Farthest-In-Future (clairvoyant algorithm)

- **Farthest-in-future.** Evict item in the cache that is not requested until farthest in the future



- **Theorem.** FF is optimal eviction schedule.
- **Pf.** Algorithm and theorem are intuitive; proof is subtle





Quiz



- Which item will be evicted next using farthest-in-future schedule?

- A
- B
- C
- D
- E

cache

	⋮	.	.	.	.
	B	D	B	Y	A
	C	D	B	C	A
	E	D	E	C	A
	F	?	?	?	?
	C				
	D				
	A				
	E				
	A				
	C				
	⋮				

request

← cache miss
(which item to eject?)





Reduced Eviction Schedules



- Def.** A **reduced** schedule is a schedule that only inserts an item d into the cache in a step j only if there is a request for d in step j and d is not already in the cache.

a	a	b	c
a	a	b	c
c	a	d	c
d	a	d	c
a	a	c	b
b	a	c	b
c	a	c	b
d	d	c	b
d	d	c	d

d enters cache
without a request

d enters cache
even though already
in cache

non-reduced schedule

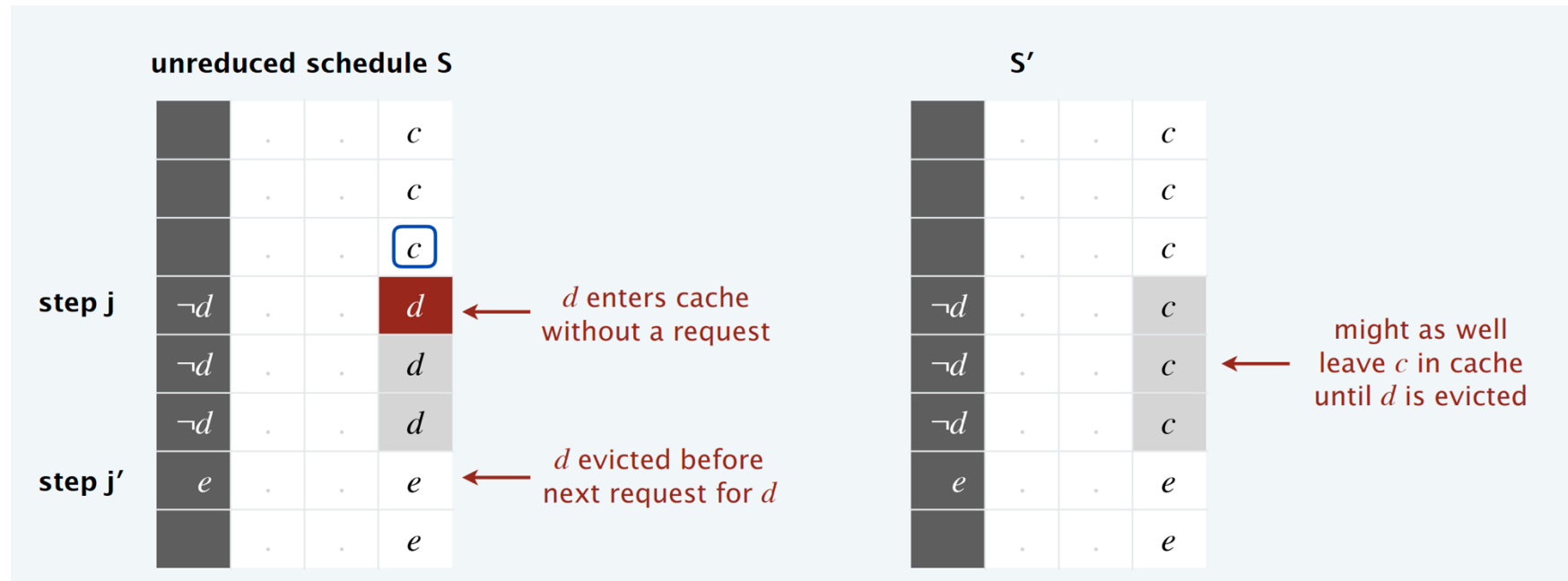
a	a	b	c
a	a	b	c
c	a	b	c
d	a	d	c
a	a	d	c
b	a	d	b
c	a	c	b
d	d	c	b
d	d	c	b

reduced schedule



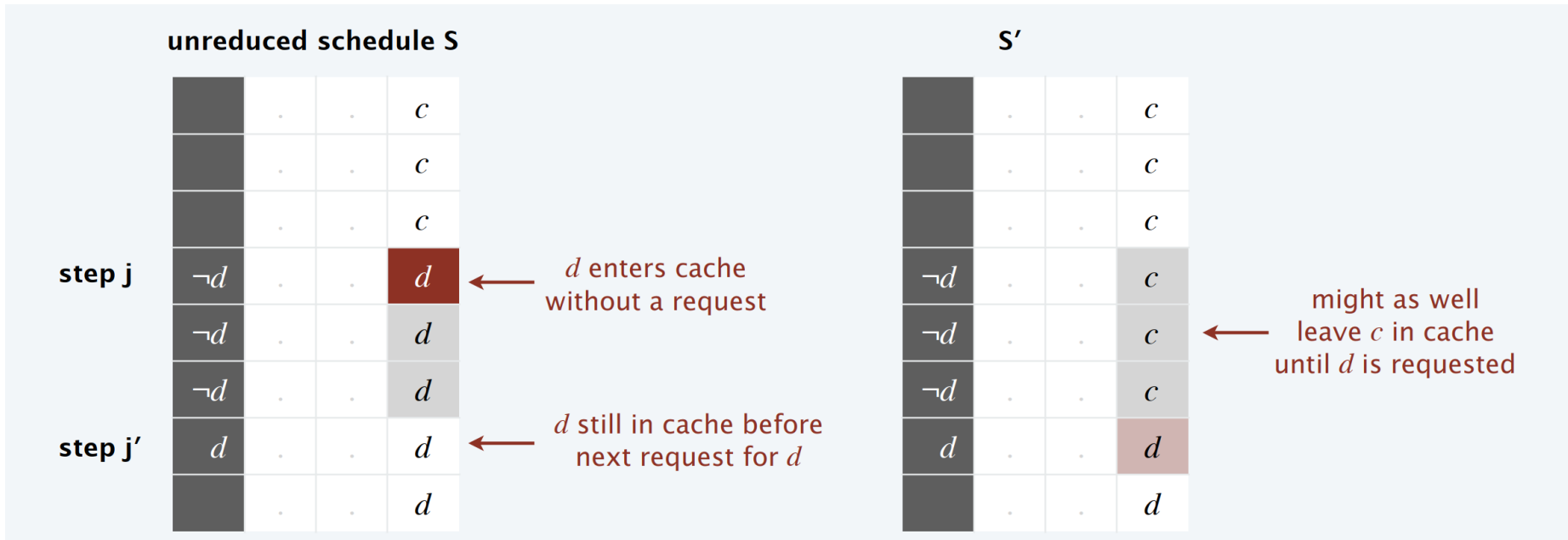
Reduced Eviction Schedules

- **Claim.** Given any unreduced schedule S , can transform it into a reduced schedule S' with no more evictions
- **Pf.** [by induction on number of steps j]
 - Suppose S brings d into the cache in step j without a request.
 - Let c be the item S evicts when it brings d into the cache.
 - Case 1a: d evicted before next request for d .



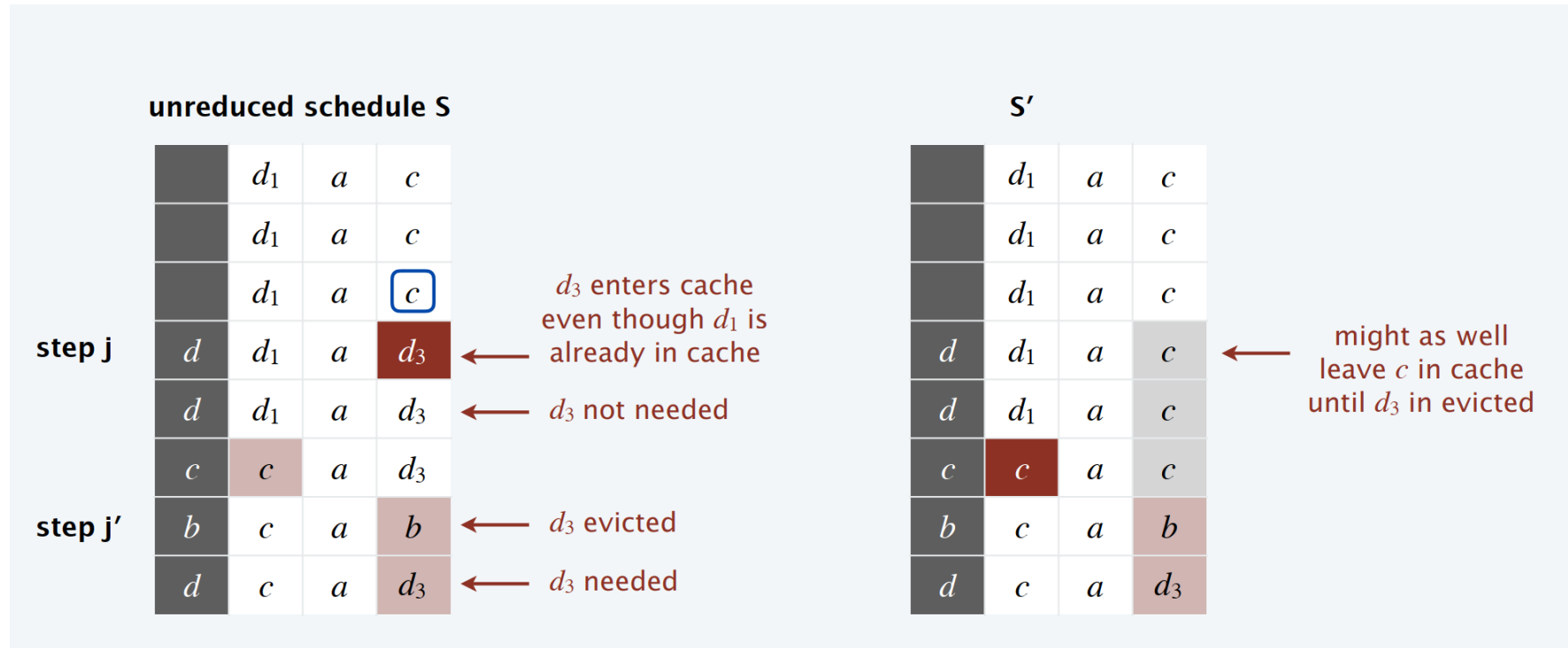
Reduced Eviction Schedules

- **Claim.** Given any unreduced schedule S , can transform it into a reduced schedule S' with no more evictions
- **Pf.** [by induction on number of steps j]
 - Suppose S brings d into the cache in step j without a request.
 - Let c be the item S evicts when it brings d into the cache.
 - Case 1a: d evicted before next request for d .
 - Case 1b: next request for d occurs before d is evicted.



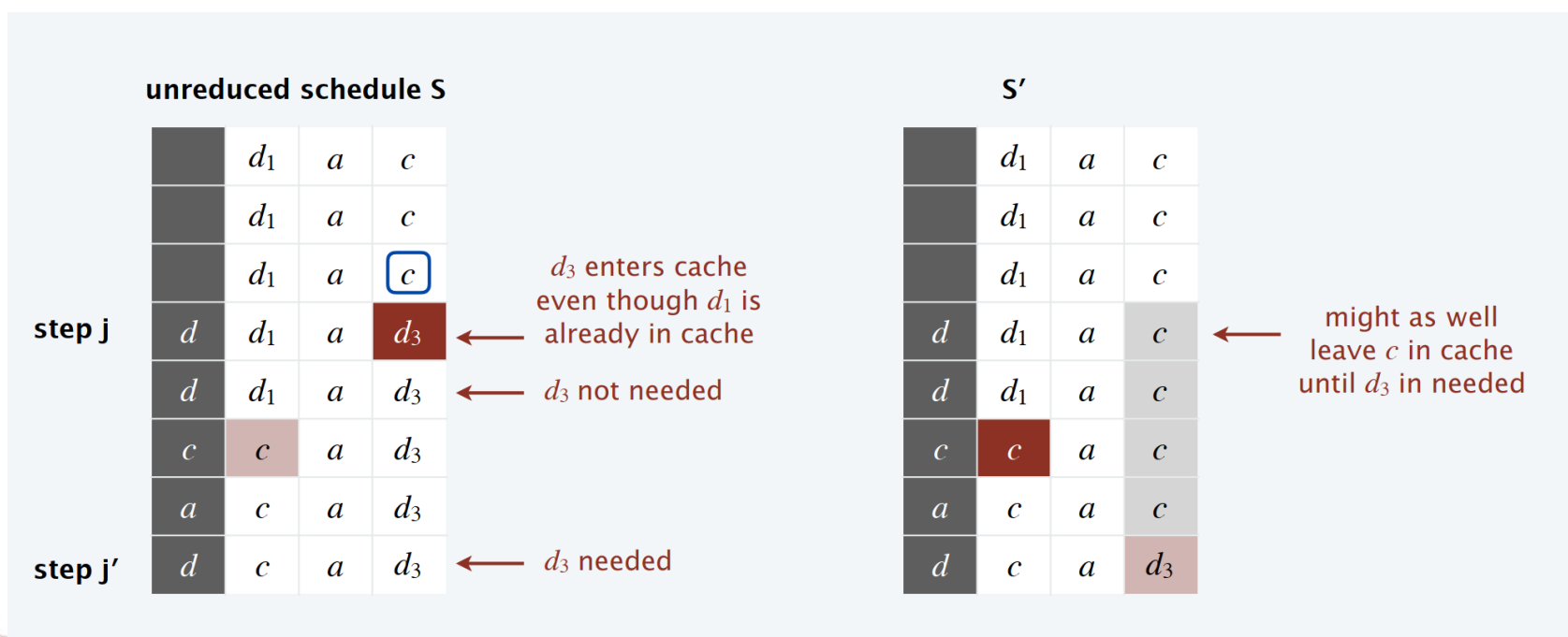
Reduced Eviction Schedules

- **Claim.** Given any unreduced schedule S , can transform it into a reduced schedule S' with no more evictions
- **Pf.** [by induction on number of steps j]
 - Suppose S brings d into the cache in step j even though d is in cache.
 - Let c be the item S evicts when it brings d into the cache.
 - Case 2a: d evicted before it is needed



Reduced Eviction Schedules

- **Claim.** Given any unreduced schedule S , can transform it into a reduced schedule S' with no more evictions
- **Pf.** [by induction on number of steps j]
 - Suppose S brings d into the cache in step j even though d is in cache.
 - Let c be the item S evicts when it brings d into the cache.
 - Case 2a: d evicted before it is needed.
 - Case 2b: d needed before it is evicted.





Reduced Eviction Schedules



- **Claim.** Given any unreduced schedule S , can transform it into a reduced schedule S' with no more evictions
- **Pf.** [by induction on number of steps j]
 - Case 1: S brings d into the cache in step j without a request. ✓
 - Case 2: S brings d into the cache in step j even though d is in cache. ✓
 - If multiple unreduced items in step j , apply each one in turn, dealing with Case 1 before Case 2.

Resolving case 1 might trigger case 2





Farthest-In-Future: Analysis

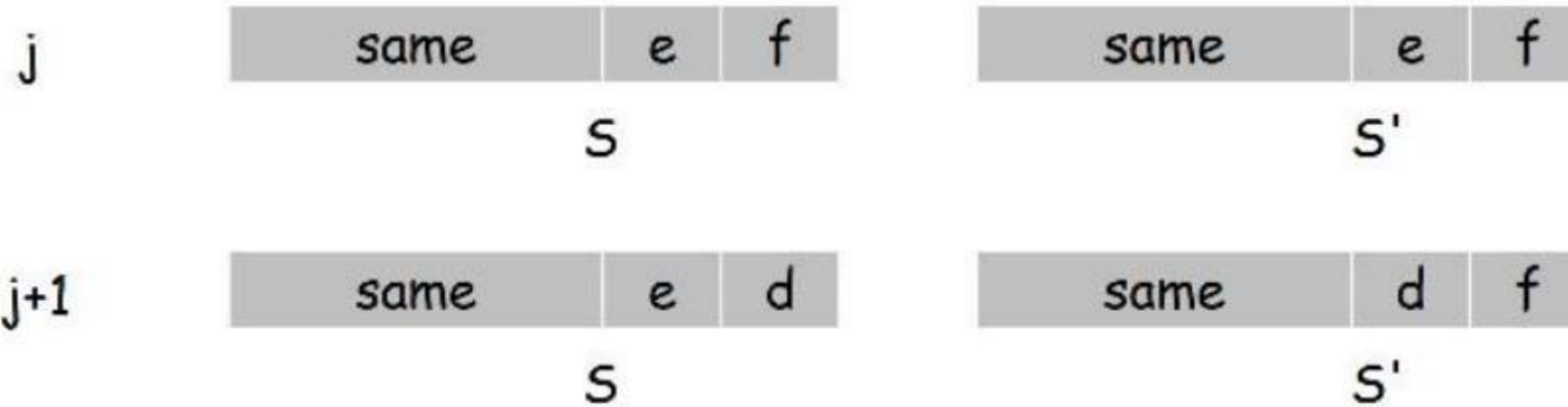
- **Lemma.** Let S be a reduced schedule that makes the same schedule as S_{FF} through the first j requests. Then there is a reduced schedule S' that makes the same schedule as S_{FF} through the first $j+1$ requests, and incurs no more eviction than S does
- **Pf.**
 - Consider $(j+1)^{st}$ request $d = d_{j+1}$
 - Since S and S_{FF} have agreed up until now, they have the same cache contents before request $j+1$
 - Case 1: (d is already in the cache). $S' = S$ satisfies invariant
 - Case 2: (d is not in the cache and S and S_{FF} evict the same element).
 $S' = S$ satisfies invariant



Farthest-In-Future: Analysis

- **Pf. (continued)**

- Case 3: (d is not in the cache; $\text{next}(e) > \text{next}(f)$)
- S_{FF} evicts e ; S evicts $f \neq e$.
- Begin construction of S' from S by evicting e instead of f



- Now S' agrees with S_{FF} on first $j+1$ requests
- From request $j+2$ onward, we make S' the same as S , but this becomes impossible when e or f is involved

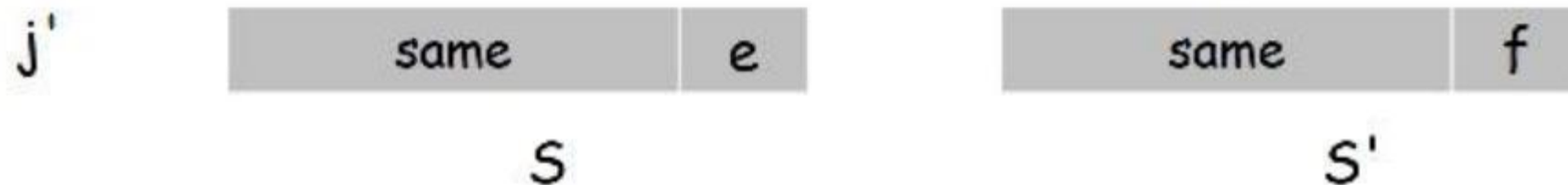


Farthest-In-Future: Analysis

- **Pf. (continued)**

- Let j' be the **first** time after $j+1$ that S and S' take a different action, and let g be item requested at time j'

must involve e or f (or both)



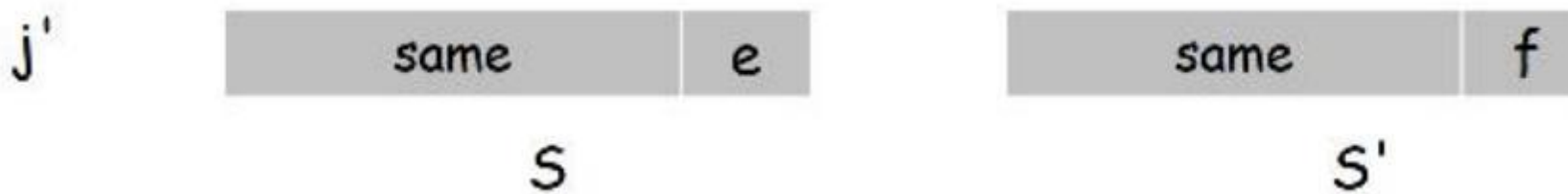
Case 3a: $g = e$. Cannot happen with Farthest-In-Future since there must be a request for f before e



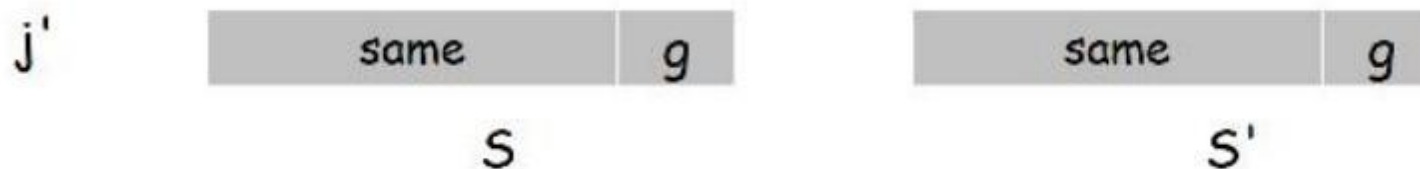


Farthest-In-Future: Analysis

- Pf. (continued)
- Let j' be the first time after $j+1$ that S and S' take a different action, and let g be item requested at time j'



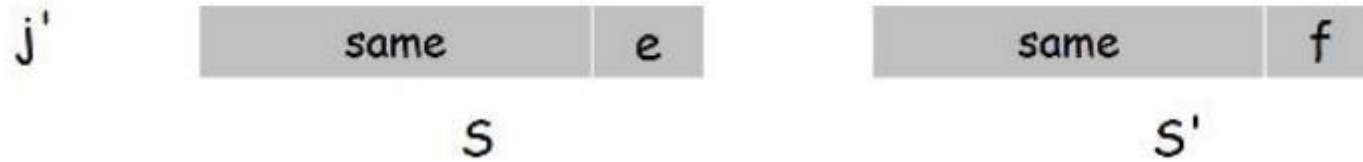
- Case 3b: $g \neq e, f$. S must evict e .
- Make S' evict f ; now S and S' have the same cache.





Farthest-In-Future: Analysis

- Pf. (continued)
- Let j' be the **first** time after $j+1$ that S and S' take a different action, and let g be item requested at time j'



- Case 3c: $g = f$. Element f cannot be in cache of S , so let e' be the element that S evicts.
 - If $e' = e$, S' accesses f from cache; now S and S' have same cache
 - If $e' \neq e$, S' evicts e' and brings e into the cache; now S and S' have the same cache.
 S' is no longer reduced, but can be transformed into a reduced schedule with
 - a) agrees with S_{FF} through step $j + 1$
 - b) has no more evictions than S





Farthest-In-Future: Analysis



- **Theorem.** FF is optimal eviction algorithm
- **Pf.** (by induction on number of requests j)
- Base case (trivial):
- There exists an optimal reduced schedule S that makes the same schedule as S_{FF} through the first **0** requests
- Inductive step (implied by the lemma):
- If there exists an optimal reduced schedule S that agrees with S_{FF} through the first **j** requests, then there exists an optimal reduced schedule S' that agrees with S_{FF} through the first **$j+1$** requests





Caching Perspective



- **Online vs. offline algorithms**
 - Offline: full sequence of requests is known as priori
 - Online (reality): requests are not known in advance
 - Caching is among most fundamental online problems in CS
- **LIFO.** Evict page brought in most recently
- **LRU.** Evict page whose most recent access was earliest
- **Theorem.** FF is optimal offline eviction algorithm.
 - Provides basis for understanding and analyzing online algorithms
 - LRU is k -competitive.
 - LIFO is arbitrarily bad





Greedy Algorithms: Summary



- **Basic idea**

- Make the locally optimal choice at each step

- **Algorithms**

- Interval Scheduling
 - Choose the job with the earliest finish time
- Scheduling to Minimize Lateness
 - Choose the job with the earliest deadline
- Clustering
 - Single-link k-clustering
- Optimal Caching
 - Evict item that is requested farthest in future

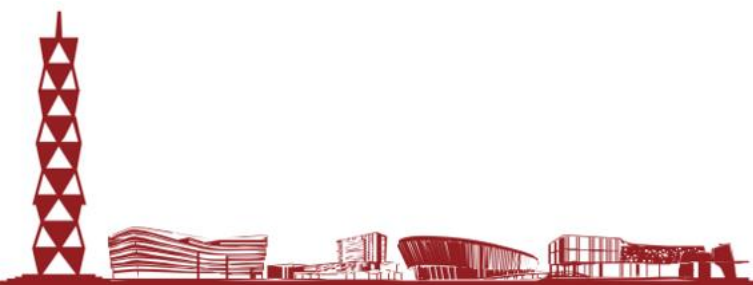
- **Proof skills**

- **Greedy algorithm stays ahead.** Show that after each step of the greedy algorithm, its solution is at least as good as any other algorithm's
- **Exchange argument.** Gradually transform any solution to the one found by the greedy algorithm without hurting its quality





Divide and Conquer





Divide-and-Conquer



- **Divide-and-conquer**

- Break up problem into several parts
- Solve each part recursively
- Combine solutions to sub-problems into overall solution

- **Most common usage**

Break up problem of size n into **two** equal parts of size $\frac{1}{2}n$ in **linear time**

Solve two parts recursively

Combine two solutions into overall solution in **linear time**

- **Consequence**

Divide-and-conquer: $\Theta(n \log n)$





Mergesort (Revisit)





Mergesort



- **Mergesort**

- Divide array into two halves
- Recursively sort each half
- Merge two halves to make sorted whole

input

A	L	G	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

sort left half

A	G	L	O	R	I	T	H	M	S
---	---	---	---	---	---	---	---	---	---

sort right half

A	G	L	O	R	H	I	M	S	T
---	---	---	---	---	---	---	---	---	---

merge results

A	G	H	I	L	M	O	R	S	T
---	---	---	---	---	---	---	---	---	---

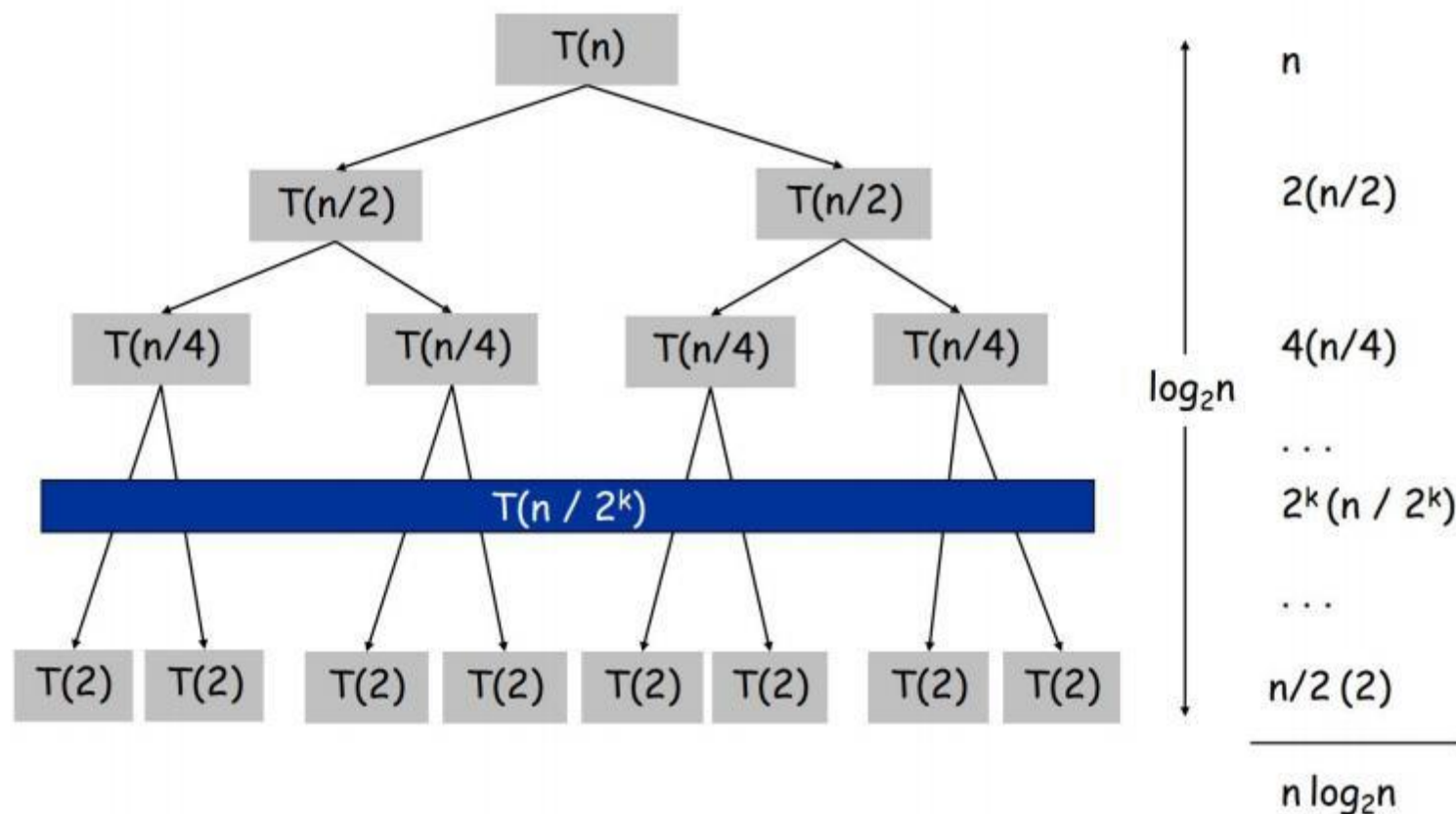




Proof by Recursive Tree



$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$



Proof by Induction

- **Claim.** If $T(n)$ satisfies this recurrence, then $T(n) = n \log_2 n$

↑
Assumes n is a power of 2

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$

- **Pf.** (by induction on n)
 - Base case: $n = 1$
 - Inductive hypothesis: $T(n) = n \log_2 n$
 - Goal: show that $T(2n) = 2n \log_2(2n)$

$$\begin{aligned} T(2n) &= 2T(n) + 2n \\ &= 2n \log_2 n + 2n \\ &= 2n(\log_2(2n) - 1) + 2n \\ &= 2n \log_2(2n) \end{aligned}$$





Closest Pair of Points





Closest Pair of Points



- **Closest pair.** Given n points in the plane, find a pair with smallest Euclidean distance between them
- **Fundamental geometric primitive**
 - Graphics, computer vision, geographic information systems, molecular modeling, air traffic control.
 - Special case of nearest neighbor, Euclidean MST, Voronoi

Fast closest pair inspired fast algorithms for these problems

- **Brute force.** Check all pairs of points p and q with $\Theta(n^2)$ comparisons
- **Assumption.** No two points have same x coordinate

To make presentation cleaner



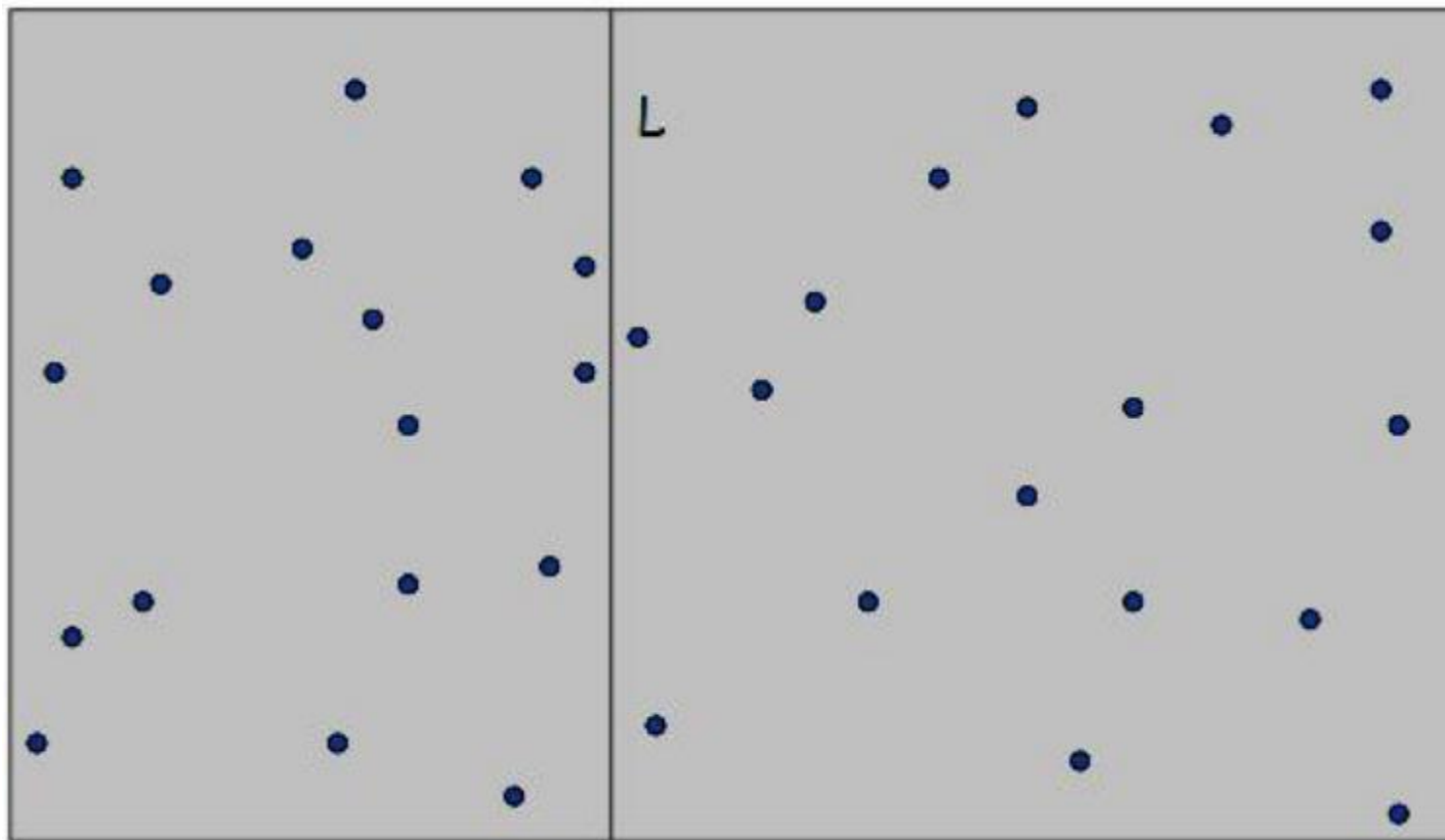


Closest Pair of Points



- **Algorithm**

- **Divide:** draw vertical line L so that roughly $\frac{1}{2}n$ points on each side



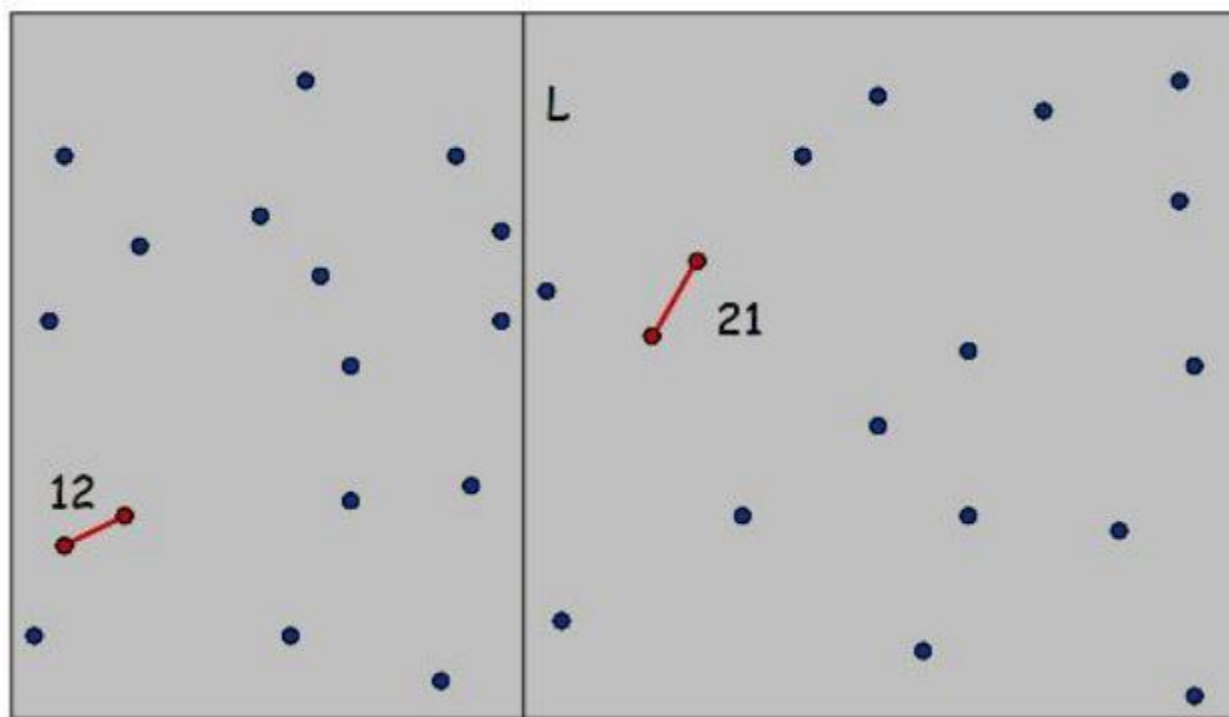


Closest Pair of Points



- **Algorithm**

- Divide: draw vertical line L so that roughly $1/2 n$ points on each side
- **Conquer:** find closest pair in each side recursively



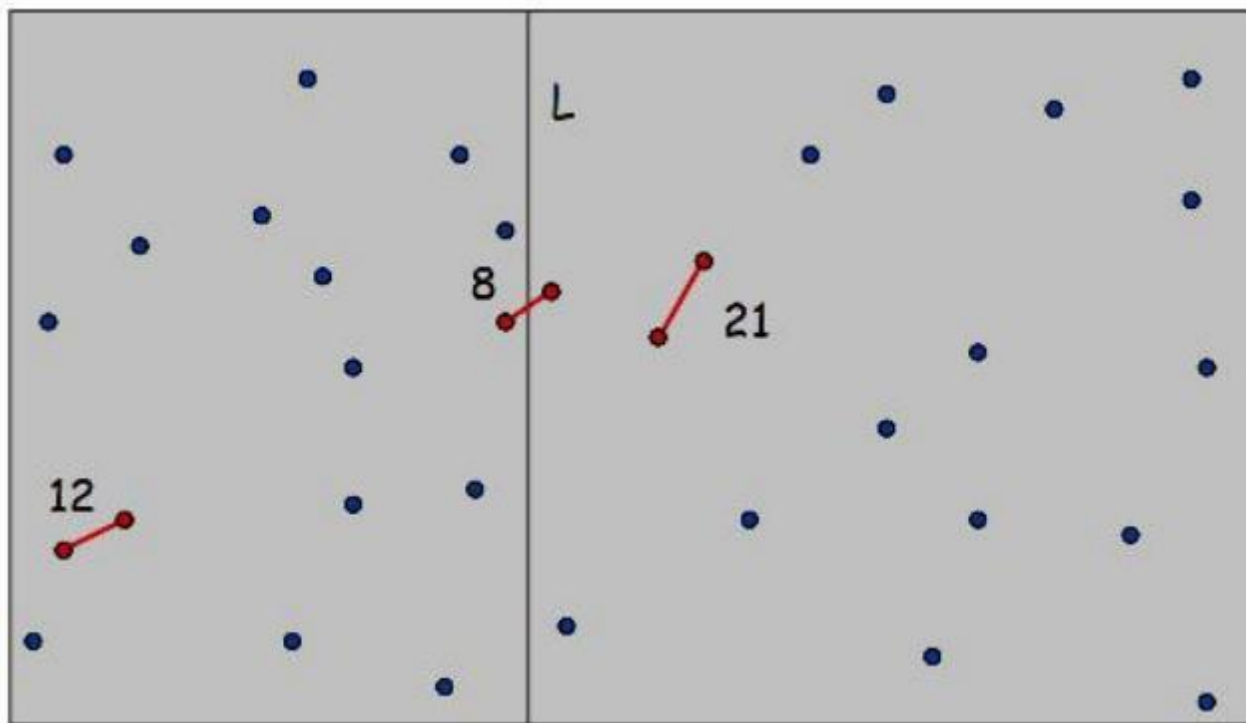


Closest Pair of Points



- Algorithm

- Divide: draw vertical line L so that roughly $\frac{1}{2}n$ points on each side
- Conquer: find closest pair in each side recursively
- **Combine:** find closest pair with one point in each side $\leftarrow \Theta(n^2)$
- Return best of 3 solutions

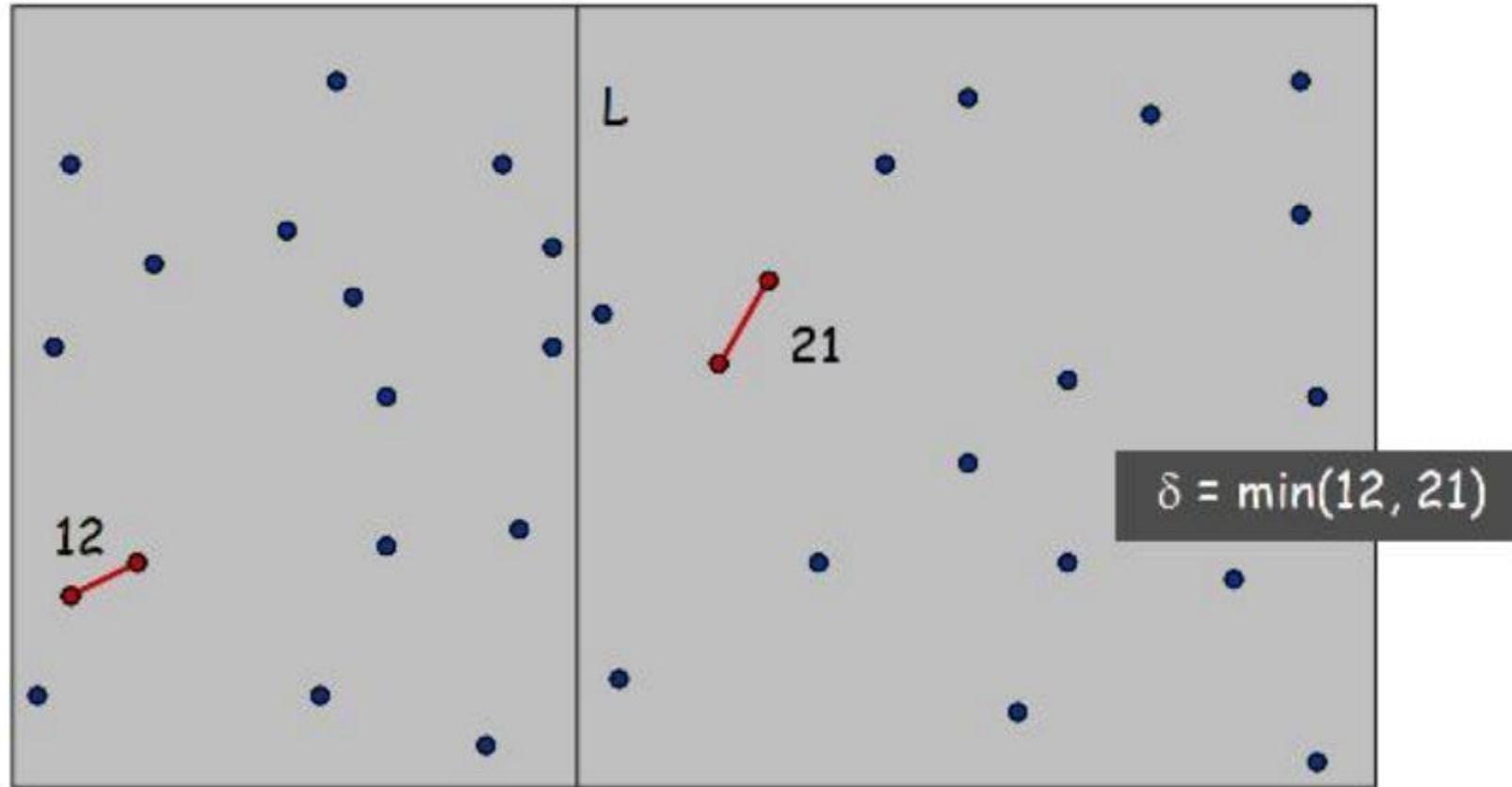




Closest Pair of Points



- Find closest pair with one point in each side, **assuming that distance $< \delta$**



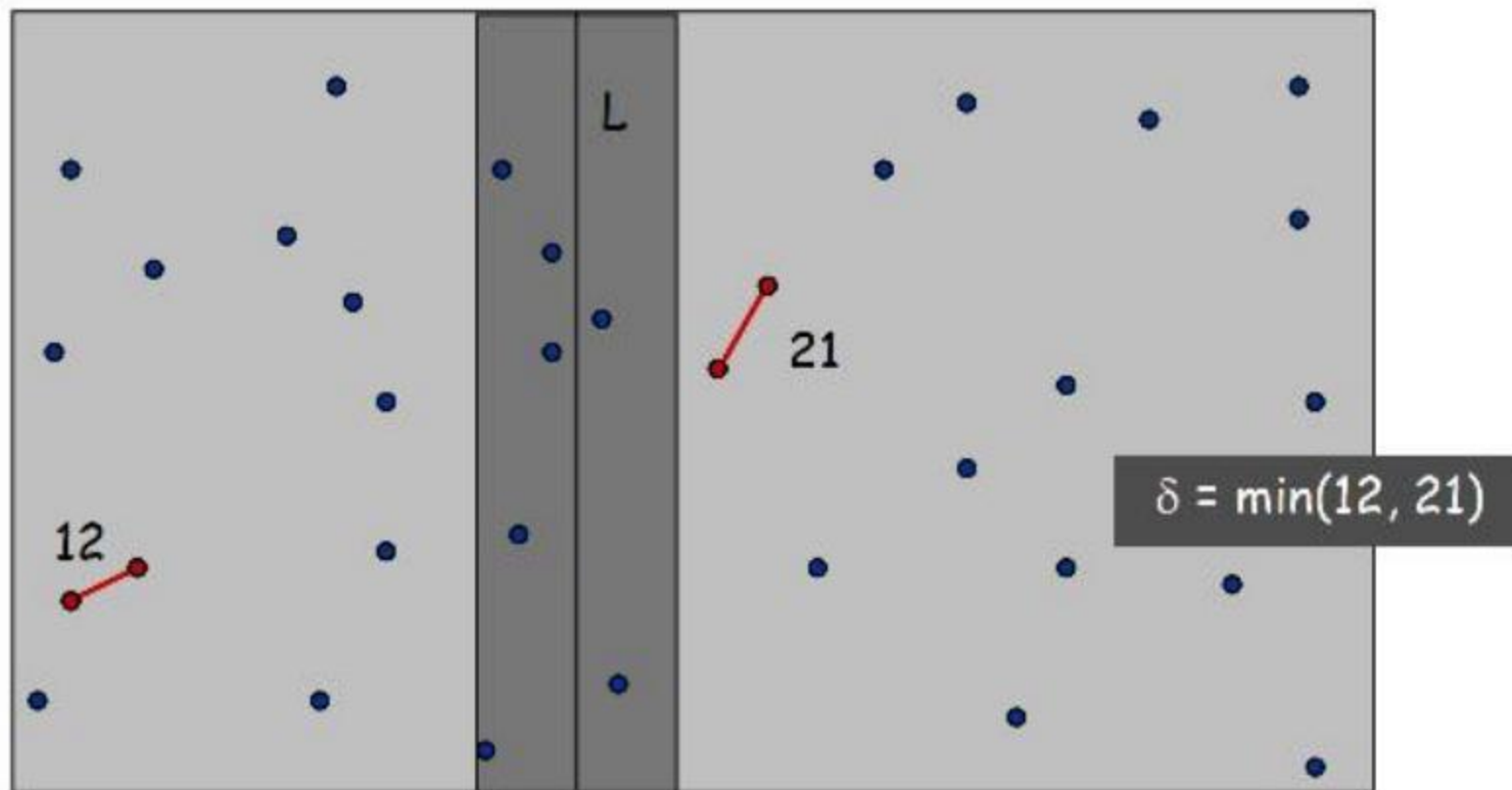


Closest Pair of Points



- Find closest pair with one point in each side, **assuming that distance $< \delta$**

Observation: only need to consider points within δ of line L





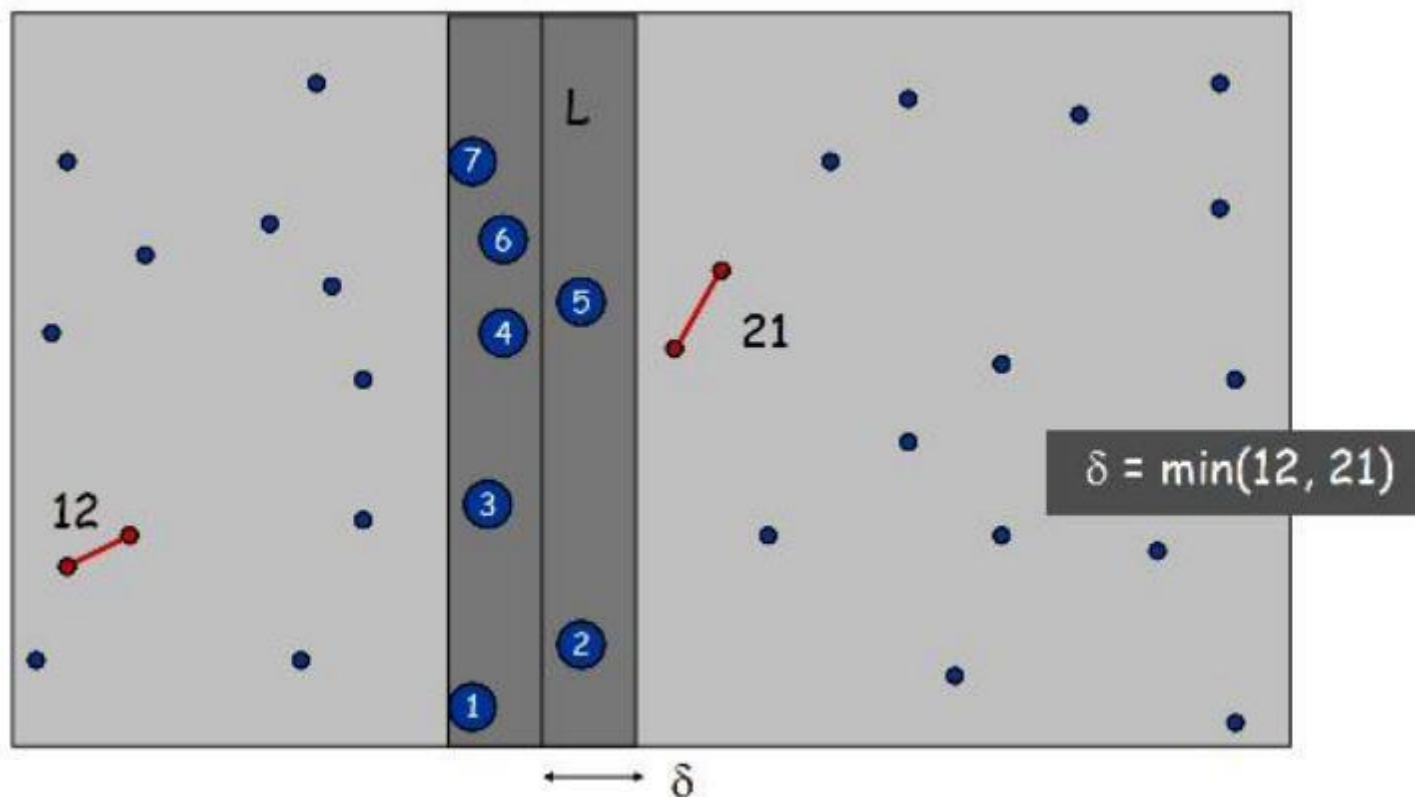
Closest Pair of Points



- Find closest pair with one point in each side, **assuming that distance $< \delta$**

Observation: only need to consider points within δ of line L

Sort points in 2δ -strip by their y coordinate





Closest Pair of Points

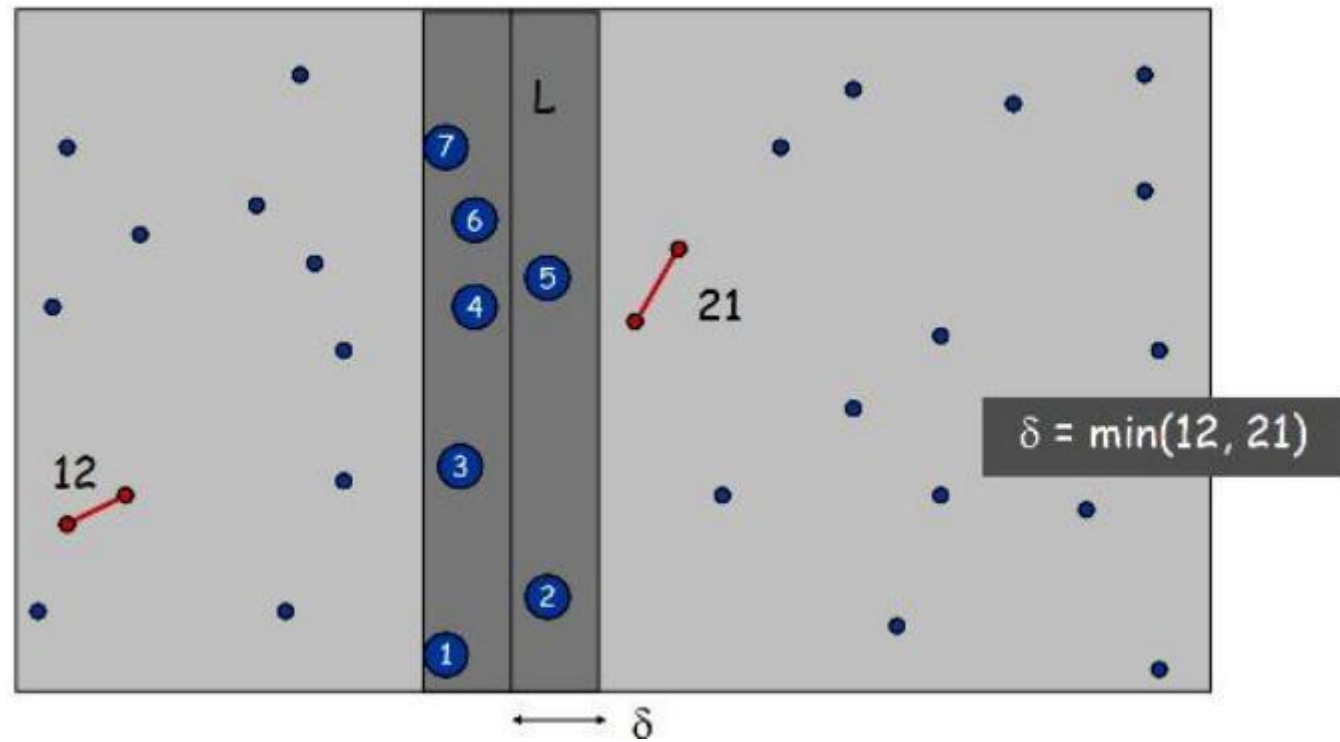


- Find closest pair with one point in each side, **assuming that distance $< \delta$**

Observation: only need to consider points within δ of line L

Sort points in 2δ -strip by their y coordinate

Only check distances of those within 11 positions in sorted list!

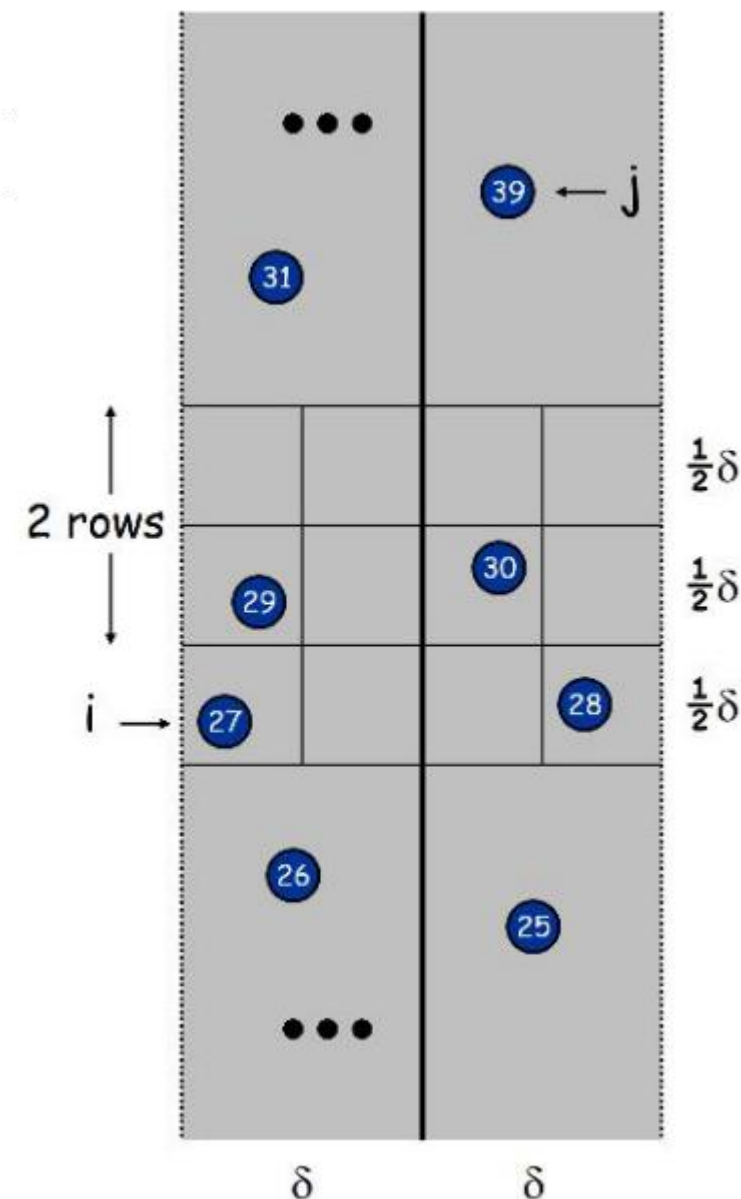




Closest Pair of Points



- **Def.** Let s_i be the point in the 2δ -strip, with the i^{th} smallest y-coordinate
- **Claim.** If $|i - j| \geq 12$, then the distance between s_i and s_j is at least δ (If $y_j - y_i \geq \delta$, then $\text{dist} \geq \delta$)
- **P-f.**
 - No two points lie in some $\frac{1}{2}\delta$ -by- $\frac{1}{2}\delta$ box
 - $2\delta / (\delta/2) = 4$, $\delta / (\delta/2) = 2$, $4 * 2 = 8$
 - Two points at least 2 rows apart have distance $\geq 2(\frac{1}{2}\delta)$

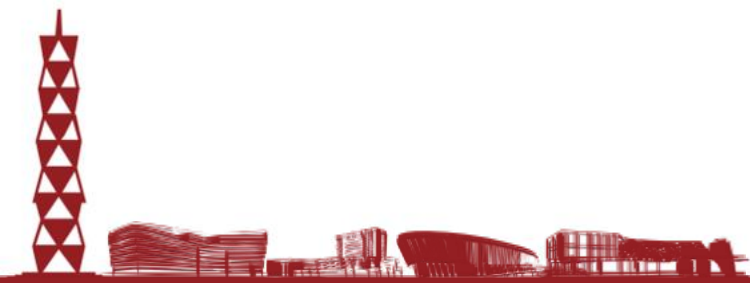




Closest Pair of Points



- Find closest pair with one point in each side, **assuming that distance $< \delta$**
 - Linear time algorithm!
- Without this assumption?
 - Run the same algorithm
 - If assumption true, we will find the right closest pair with one point in each side
 - If false, the algorithm will find a pair with distance $\geq \delta$, and then the combine step will correctly return δ as distance of closest pair





Closest Pair Algorithm



```
Closest-Pair( $p_1, \dots, p_n$ ) {
```

```
  Compute separation line  $L$  such that half the points  
  are on one side and half on the other side.
```

$O(n \log n)$

```
   $\delta_1 = \text{Closest-Pair}(\text{left half})$ 
```

```
   $\delta_2 = \text{Closest-Pair}(\text{right half})$ 
```

$2T(n / 2)$

```
   $\delta = \min(\delta_1, \delta_2)$ 
```

```
  Delete all points further than  $\delta$  from separation line  $L$ 
```

$O(n)$

```
  Sort remaining points by y-coordinate.
```

$O(n \log n)$

```
  Scan points in y-order and compare distance between  
  each point and next 11 neighbors. If any of these  
  distances is less than  $\delta$ , update  $\delta$ .
```

$O(n)$

```
  return  $\delta$ .
```

```
}
```





Closest Pair of Points: Analysis



- Running time

$$T(n) \leq 2T(n/2) + O(n \log n) \Rightarrow T(n) = O(n \log^2 n)$$

- Q. Can we achieve $O(n \log n)$?
- A. Yes. Don't sort points from scratch each time.
 - Sort all the points twice before recursive call, once by x coordinate and once by y coordinate
 - Reuse the sorted sequences when needed (linear time)

$$T(n) \leq 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$





Next Time: Divide and Conquer (Cont.)

